

1 / 2 · 양자화 / 역양자화

실수↔정수 대응식과 layer 출력의 scale 보정을 복원한다.

실행: float → /scale → round → +zero → clamp | 역추적: float 근사 ← (정수-zero)×scale

[기출 복기 관련 + 범위 핵심] 기본 양자화

```
scale = (fp_max - fp_min) / (quantized_max - quantized_min)
zero_point = round(quantized_min - fp_min / scale)
scaled_tensor = fp_tensor / scale
rounded_tensor = torch.round(scaled_tensor)
rounded_tensor = rounded_tensor.to(dtype)
shifted_tensor = rounded_tensor + zero_point
# 범위를 벗어난 정수는 quantized_min/max로 clamp
# 역양자화 일반식: 먼저 실수형으로 계산
fp_reconstructed = (quantized_tensor.float() - zero_point) * scale
```

인자 출처 fp_tensor-bitwidth-scale-zero_point-dtype: linear_quantize 입력. fp_min/max: fp_tensor.min/max.
quantized_min/max: bitwidth와 signed 여부로 계산. scale와 zero_point 계산 함수의 앞부분을 확인.

실수 방지 signed b-bit 범위 $[-2^{b-1}, 2^{b-1}-1]$. round는 정수 값으로 반올림하되 dtype 변환은 별도. torch.round의 정확한 .5는 가까운 짝수. 10~20을 1000~2000에 대응시키면 scale=.01, zero_point=0.

근거: 2. Quantization for CNN_answer.ipynb · get_quantization_scale_and_zero_point / linear_quantize; 역양자화는 공식식

[기출 복기] quantized_linear / quantized_conv2d

```
# weight scale을 output 채널에 맞게 broadcast
weight_scale = weight_scale.flatten().view(1, -1) # Linear
# Conv2d일 때는 .flatten().view(1, -1, 1, 1)
# 정수 누산 또는 이를 흉내 낸 연산 후
output = output.float()
output = output * (input_scale * weight_scale / output_scale)
output = output + output_zero_point
output = output.round().clamp(
    *get_quantized_range(feature_bitwidth)).to(torch.int8)
# bias를 미리 양자화할 때: bias_scale = input_scale * weight_scale
```

인자 출처 input_scale-weight_scale-output_scale-output_zero_point: layer 함수 인자, 앞서 입력/가중치/출력을 양자화하며 정한 값. output: F.linear/F.conv2d 누산 결과. reshape 기준: 출력 채널 축.

실수 방지 input×weight를 곱했으므로 실제 단위는 두 scale의 곱. output_scale로 나눠 출력 정수 단위로 맞춥니다.
zero-point 보정이 bias에 흡수됐는지 앞 코드를 확인하여 두 번 빼지 않습니다. K-means의 대표값 수는 $2^{bitwidth}$.

근거: 2. Quantization for CNN_answer.ipynb · quantized_linear / quantized_conv2d / bias quantize / k_means_quantize

가리고 쓰기: scale의 분자/분모 / zero_point 부호 / round / input_scale*weight_scale/output_scale.

2 / 2 · Pruning과 Distillation

작은 중요도 가중치를 제거하고 teacher-정답을 student 손실에 연결한다.

Pruning: 중요도 → threshold → mask | Distillation: teacher/student → soft loss + label loss

[범위 핵심] Magnitude pruning

```
num_pruned_elements = round(weight.numel() * sparsity)
importance = torch.abs(weight)
threshold = torch.kthvalue(importance.flatten(), num_pruned_elements)[0]
mask = importance > threshold
weight.mul_(mask)
# LLM의 activation-aware 중요도 (별도 방법)
importance = torch.abs(W) * input_feat[n]
```

인자 출처 weight-sparsity: pruning 함수 입력. numel은 전체 원소 수. threshold: k번째 작은 중요도. W: 현재 Linear weight, input_feat[n]: calibration에서 수집한 현재 layer 입력 특성 통계.

실수 방지 mask=True가 남길 가중치. k=0 또는 sparsity=1은 앞의 경계 처리부터 확인. threshold 동물이면 정확히 k개보다 많이 제거될 수 있음. 0을 만드는 것만으로 실제 연산 시간이 반드시 줄지는 않습니다.

근거: 1. Pruning for CNN_answer.ipynb · fine-grained pruning / 4. Pruning for LLM_answer.ipynb

[범위 핵심] teacher + 정답 → student 학습

```
teacher.eval()
student.train()
with torch.no_grad():
    teacher_logits = teacher(inputs)
    student_logits = student(inputs)
    soft_targets = nn.functional.softmax(teacher_logits / T, dim=-1)
    student_prob = nn.functional.softmax(student_logits / T, dim=-1)
    soft_targets_loss = torch.sum(
        soft_targets * (soft_targets.log() - student_prob.log())
    ) / student_prob.size(0) * (T**2)
    label_loss = ce_loss(student_logits, labels)
    loss = (soft_target_loss_weight * soft_targets_loss
            + ce_loss_weight * label_loss)
    loss.backward()
    optimizer.step()
```

인자 출처 teacher-student-T-두 loss weight: train_knowledge_distillation 입력. inputs,labels: loader 배치. ce_loss=CrossEntropyLoss. optimizer는 student.parameters()로 생성, backward 전에 zero_grad.

실수 방지 soft 분포는 teacher와 student 둘 다 T로 나눔, 마지막 클래스 축 softmax. label_loss에는 student logits와 진짜 labels. 위는 원본 직접 KL 식; KLDivLoss를 쓰는 변형은 첫 입력에 log_softmax, target에 teacher softmax, T**2 보정.

근거: 3. Knowledge Distillation_answer.ipynb · 셀 27 train_knowledge_distillation

가리고 쓰기: abs→kthvalue→mask / teacher만 no_grad / T가 들어가는 세 위치 / student optimizer.